

Virtualisation, Dockerisation Simulation & Émulation

Comprendre les concepts, architectures et usages



Sommaire

01



Virtualisation

Type 1 (bare-metal) vs Type 2, hyperviseurs, consommation CPU/RAM

02



Dockerisation

Conteneurs, noyau partagé, orchestration, cas d'usage entreprise

03



Simulation

Modélisation logicielle, environnements virtuels, usages scientifiques

04



Émulation

Reproduction matérielle, rétrocompatibilité, cas d'usage

05



Comparatif

Tableau récapitulatif, consommation, performances, choix



VIRTUALISATION

Créer des machines virtuelles sur une seule machine physique

Qu'est-ce que la Virtualisation ?



Définition

La virtualisation permet de créer plusieurs machines virtuelles (VM) sur un seul serveur physique.

Chaque VM possède :

- Son propre OS complet
- Ses ressources dédiées (CPU, RAM, disque)
- Son isolation totale des autres VM
- Ses propres pilotes et bibliothèques



L'Hyperviseur

L'hyperviseur est la couche logicielle qui crée et gère les VM.

Il existe deux types :

Type 1 — Bare-metal

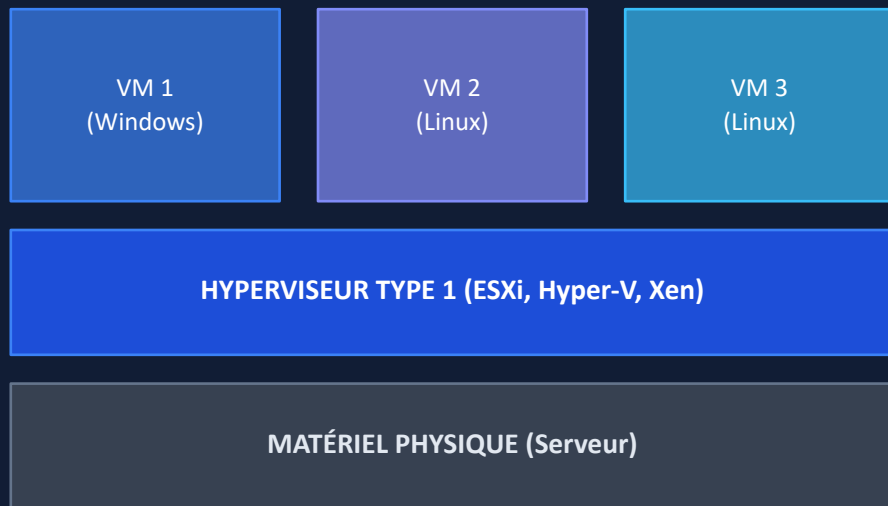
Directement sur le matériel

Type 2 — Hébergé

Sur un OS existant (hôte)

Virtualisation Type 1 — Bare-Metal

Architecture



Caractéristiques

- Pas d'OS hôte entre l'hyperviseur et le matériel
- Performances proches du natif
- Utilisé en datacenter & production

Exemples :

- VMware ESXi
- Microsoft Hyper-V
- Citrix Xen Server
- KVM (Linux)

Virtualisation Type 2 — Hébergée

Architecture



Caractéristiques

- S'installe SUR un OS existant
- Plus simple à mettre en place
- Performances réduites (couche OS supplémentaire)
- Usage : développement, test, formation

Exemples :

- Oracle VirtualBox
- VMware Workstation
- Parallels Desktop

Consommation CPU / RAM — Virtualisation



Type 1 — Bare-Metal

CPU Overhead



2-5%

RAM Overhead



~256 Mo par VM + OS client

- Accès direct au matériel
- Overhead minimal
- RAM : OS complet par VM (1-8 Go typique)



Type 2 — Hébergé

CPU Overhead



10-20%

RAM Overhead



OS hôte + ~512 Mo/VM + OS
CLIENT

- Couche OS hôte = overhead supplémentaire
- Performances dégradées sous charge
- RAM : OS hôte + OS invité par VM



DOCKERISATION

Conteneurs légers partageant le noyau de l'hôte

Qu'est-ce que la Dockerisation ?



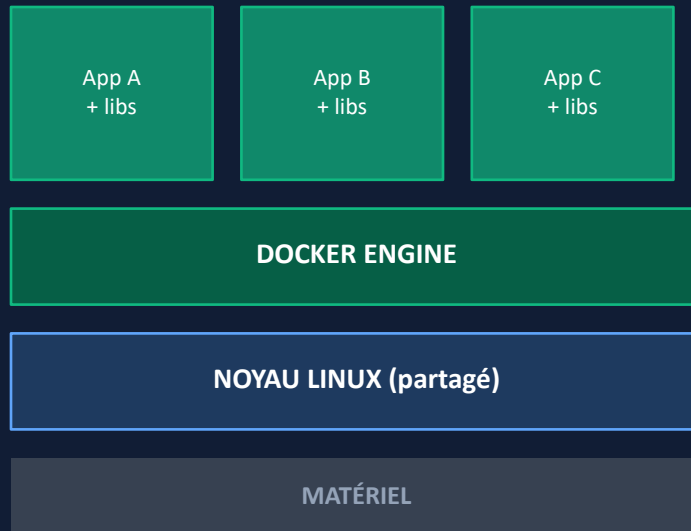
Principe

Un conteneur est une unité logicielle légère qui empaquète une application avec toutes ses dépendances.

Différence clé avec une VM :

- Pas d'OS invité complet
- Partage du noyau (kernel) de l'hôte
- Démarrage en secondes (vs minutes)
- Image de quelques Mo (vs Go)

Noyau partagé



Le Noyau Linux et Docker



Namespaces

Isolation des processus, réseau, utilisateurs, montages



Cgroups

Limitation CPU, RAM, I/O par conteneur



Union FS

Système de fichiers en couches (overlay2)



Seccomp / AppArmor

Filtrage des appels système, sécurité renforcée

Docker en Entreprise — Cas d'Usage



Microservices

Découpage d'applications monolithiques en services indépendants et scalables



CI/CD Pipeline

Builds reproductibles, tests isolés, déploiement continu automatisé



Cloud & Kubernetes

Orchestration de centaines de conteneurs avec K8s, scaling automatique



Dev / Prod Parity

Même image du développement à la production, fini le "ça marche chez moi"



Bases de données

PostgreSQL, Redis, MongoDB en conteneurs pour le dev et le test



Edge & IoT

Déploiement léger sur des dispositifs embarqués et passerelles

Consommation CPU / RAM — Docker

<1%

CPU Overhead

Exécution quasi-native grâce au noyau partagé

~10 Mo

RAM par conteneur

Pas d'OS invité, uniquement les libs de l'app

<1s

Temps de démarrage

Conteneur opérationnel en millisecondes

Pourquoi si léger ?

- Pas de noyau dupliqué : tous les conteneurs partagent le kernel Linux de l'hôte
- Pas d'OS invité : seuls les binaires et bibliothèques nécessaires sont inclus
- Un serveur peut héberger des centaines de conteneurs vs quelques dizaines de VM



SIMULATION

Modéliser un système par le logiciel

Qu'est-ce que la Simulation ?



Définition

La simulation reproduit le comportement d'un système réel à l'aide d'un modèle logiciel.

Elle ne reproduit PAS le matériel, mais les résultats attendus.

- Modèles mathématiques
- Comportements approximatifs
- Résultats proches du réel
- Plus rapide que l'émulation

Exemples

- **Simulateurs de vol**

Flight Simulator, X-Plane

- **Simulation réseau**

GNS3, Packet Tracer, NS-3

- **Simulation scientifique**

Météo, aérodynamique, molécules

- **Jeux vidéo**

Physique, fluides, IA

Consommation CPU / RAM — Simulation

Variable selon la complexité du modèle

- CPU : de léger (Packet Tracer) à extrême (simulation CFD, météo) — peut nécessiter des clusters HPC
- RAM : de 100 Mo (simulation simple) à des centaines de Go (simulation moléculaire, IA)

Spectre de complexité

Packet Tracer
Simulation réseau

CPU: Faible

RAM: ~200 Mo

GNS3
Simulation avancée

CPU: Moyen

RAM: 2-8 Go

CFD / Météo
HPC Scientifique

CPU: Extrême

RAM: 64-512 Go



ÉMULATION

Reproduire fidèlement un matériel cible

Qu'est-ce que l'Émulation ?



Définition

L'émulation reproduit le fonctionnement d'un matériel réel de manière logicielle.

Chaque instruction du processeur cible est traduite pour le processeur hôte.

- Reproduction exacte du matériel
- Compatibilité binaire totale
- Très gourmand en CPU
- Plus lent que le matériel d'origine

Exemples

- **Consoles de jeux**

RetroArch, Dolphin, PCSX2, RPCS3

- **Processeurs / Architecture**

QEMU (ARM sur x86, RISC-V)

- **Mobiles**

Android Emulator, BlueStacks

- **Systèmes anciens**

DOSBox, Wine (couche compatibilité)

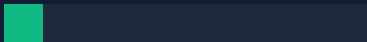
Consommation CPU / RAM — Émulation

L'émulation est la technologie la plus gourmande en CPU — chaque instruction matérielle est traduite en temps réel

Émulation 8/16-bit
(NES, SNES, Game Boy)

CPU: <5%

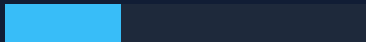
RAM: ~50 Mo



Émulation 32/64-bit
(PS1, N64, DS)

CPU: 10-30%

RAM: 200-500 Mo



Émulation moderne
(PS2, GameCube, Wii)

CPU: 40-70%

RAM: 1-4 Go



Émulation récente
(PS3, Switch, Android)

CPU: 80-100%

RAM: 4-16 Go



Tableau Récapitulatif

Critère	VM Type 1 (Bare-Metal)	VM Type 2 (Hébergée)	Docker (Conteneur)	Simulation	Émulation
Principe	Hyperviseur sur matériel	Hyperviseur sur OS hôte	Conteneur sur noyau partagé	Modèle logiciel du comportement	Reproduction du matériel
CPU Overhead	2-5%	10-20%	<1%	Variable (léger → extrême)	20-100%
RAM	1-8 Go / VM + ~256 Mo overhead	OS hôte + 1-8 Go / VM	~10 Mo par conteneur	100 Mo à 512 Go	50 Mo à 16 Go
Démarrage	30s - 2 min	1 - 5 min	<1 seconde	Instantané à plusieurs min	1 - 30s
Isolation	Totale (OS complet)	Totale (OS complet)	Partielle (noyau partagé)	N/A	Totale (matériel virtuel)
Usage principal	Production Datacenter	Dev, Test Formation	Microservices CI/CD, Cloud	Recherche Modélisation	Rétro-compat. Dev embarqué

En Résumé



Virtualisation

Isolation forte, OS complets,
idéal production



Docker

Léger, rapide, noyau partagé,
microservices



Simulation

Modélise le comportement,
pas le matériel



Émulation

Reproduit le matériel,
gourmand en CPU

Le choix dépend de votre besoin : performance, isolation, portabilité ou compatibilité.

Merci pour votre attention !